



AJAX

Programación con JavaScript II



Actividad de Bienestar





Temas

- | | |
|---|---|
| 1 | Manejadores de paquetes y empaquetados de módulos |
| 2 | Nuevos tipos y características |
| 3 | Asincronía |
| 4 | API |
| 5 | Ajax |
| 6 | Scope y This <input type="text"/> |
| 7 | Transpiladores |
| 8 | Pruebas |



Sesión	Tema	Conexión
S3	 Asincronía	Call Stack, Web APIs, Callback Queue, Promises, <code>async/await</code>
S4	 API	Qué es una API, consumir endpoints, JSON, XMLHttpRequest, <code>fetch()</code>
S5	 AJAX	Comunicación asíncrona cliente-servidor sin recargar la página



Prework

- Leer detenidamente y comprender el material explicado en el tema **Ajax**.
- Practicar todos los ejemplos que se describen previamente en el tema **Ajax**.
- Revisar cada uno de los recursos adicionales que se proponen en el tema **Ajax**.



Para saber más

Lecturas

- MDN. (2022). Fetch API. Recuperado de https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- Firthous. (2020). The Ultimate JavaScript Fetch API Cheatsheet. Recuperado de <https://javascript.plainenglish.io/the-ultimate-javascript-fetch-api-cheatsheet-e60b98c8cdbe>



Para saber más

Videos

- Vida MRR – Programación Web. (2018, 21 de noviembre). Diferencia entre Fetch vs AJAX [Archivo de video]. Recuperado de https://www.youtube.com/watch?v=V_acKBZ1ZPU
- ProCode TV. (2018, 26 de noviembre). Fetch API - #JavaScript - Consumir API Publica con JavaScript [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=cjrt3NilcNw>
- Leonidas Esteban. (2019, 30 de agosto). Fetch API en JavaScript (GET, POST, PUT, DELETE) | programación asíncrona [Archivo de video]. Recuperado de <https://www.youtube.com/watch?v=aKPcs-ElzZI>



Actividad Guiada

Parte 1

Duración: 75 minutos.



¿Qué es AJAX?

AJAX (Asynchronous JavaScript and XML) es un conjunto de técnicas que permite a una aplicación web **enviar y recibir datos del servidor en segundo plano**, logrando que solo se actualicen partes específicas del sitio sin interrumpir la interacción del usuario.



Ventajas de AJAX

- Actualiza solo los elementos necesarios.
- Mejora la velocidad y la experiencia del usuario.
- No interrumpe la interfaz gráfica.
- Simula el comportamiento de una app de escritorio.



Componentes clave

- **XMLHttpRequest**: objeto que permite la comunicación asíncrona.
- **JavaScript/DOM**: para procesar y mostrar la información.
- **CSS**: para mantener el estilo visual.
- **XML o JSON**: para estructurar los datos.



API Fetch

Es una alternativa moderna y más sencilla a XMLHttpRequest. Permite hacer peticiones HTTP usando promesas, mejorando la legibilidad y eficiencia del código.

```
fetch('archivo.json')  
  .then(response => response.json())  
  .then(data => console.log(data));
```



Caso de uso de AJAX

Pregunta:

 *¿Cuándo usarías AJAX en una aplicación web?*

Respuesta:

 Cuando necesitemos actualizar datos sin recargar toda la página.

 *Ejemplo:* Actualizar el clima de una ciudad seleccionada en un formulario.



¿Por qué elegir Fetch sobre XHR?

Ventajas de Fetch:

- Sintaxis más limpia y moderna
- Uso de promesas
- Mejor manejo de errores
- Menos código



¿Por qué usar .catch() en Fetch?

Importancia del método .catch():

- Captura errores de red o fallos del servidor
- Previene que la app se caiga
- Permite mostrar mensajes personalizados al usuario

```
.catch(error => {  
    console.error('Error:',  
error);  
});
```



Ejercicio 1: AJAX y Fetch

Objetivo del ejercicio:

Analizar y aplicar los fundamentos de **AJAX** en **JavaScript** mediante el uso del objeto **XMLHttpRequest** y la **API Fetch**, comparando sus ventajas, sintaxis y casos de uso, para comprender cómo realizar solicitudes asíncronas al servidor sin recargar la página.



Completa el código XMLHttpRequest

```
function CargaDocumento() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("demo").innerHTML = this.responseText;  
        }  
    };  
    xhttp.open("GET", "ajax_info.txt", true);  
    xhttp.send();  
}
```



Completa el código XMLHttpRequest

```
function CargaDocumento() {  
    var xhttp = new XMLHttpRequest();  
    xhttp.onreadystatechange = function() {  
        if (this.readyState == 4 && this.status == 200) {  
            document.getElementById("demo").innerHTML = this.responseText;  
        }  
    };  
    xhttp.open("GET", "ajax_info.txt", true);  
    xhttp.send();  
}
```



Reescribe usando Fetch

! TRUE = FALSE

```
function CargaDocumento() {  
  fetch('ajax_info.txt')  
    .then(res => {  
      if (!res.ok) throw new Error('Archivo no disponible');  
      return res.text();  
    })  
    .then(data => {  
      document.getElementById("demo").innerHTML = data;  
    })  
    .catch(err => {  
      document.getElementById("demo").innerHTML = "Error al cargar el contenido.";  
    });  
}
```



Reescribe usando Fetch

```
function CargaDocumento() {  
  fetch('ajax_info.txt')  
    .then(res => {  
      if (!res.ok) throw new Error('Archivo no disponible');  
      return res.text();  
    })  
    .then(data => {  
      document.getElementById("demo").innerHTML = data;  
    })  
    .catch(err => {  
      document.getElementById("demo").innerHTML = "Error al cargar el contenido."  
    });  
}
```



Comparación Fetch vs XHR

Característica	XMLHttpRequest	fetch()
Sintaxis	Más verbosa, basada en eventos/callbacks	Más limpia, basada en Promesas
Manejo de errores	Manual (hay que revisar status)	Más claro con .catch()
Lectura del cuerpo	responseText, responseXML	.text(), .json(), etc.
Código moderno (ES6+)	✗	✓



Conclusión

- AJAX mejora la experiencia del usuario con respuestas rápidas.
- XMLHttpRequest es útil, pero Fetch es más moderno.
- El manejo de errores es esencial en aplicaciones reales.
- Ambos métodos permiten cargar datos sin recargar la página.



Ejercicio 2: Uso de Fetch con Live Server

Objetivo del ejercicio:

Implementar y ejecutar una solicitud `fetch()` en un entorno local simulado mediante Live Server, con el propósito de cargar y visualizar datos dinámicos desde una API pública en una tabla HTML utilizando JavaScript.



¿Qué es Live Server?

Live Server es una extensión de VS Code que:

- Sirve archivos HTML en un entorno local.
- Permite ver los cambios en tiempo real.
- Emula el comportamiento de un servidor web real.

👉 Útil para trabajar con `fetch()` ya que muchas funciones AJAX requieren ejecutarse desde un servidor, no solo desde el sistema de archivos (`file://`).



Live Server – Visual Studio

- Descarga la extensión Live Server para Visual Studio Code desde la página oficial de Visual Studio
- Crea un nuevo proyecto `ActividadGuiadaTemaAjax`.
- Dentro del proyecto crea el archivo `index.html`.
- Escribe el siguiente código.



Código

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.0-beta3/dist/js/bootstrap.bundle.min.js"
  integrity="sha384-JEW9xMcG8R+pH31jmWH6WP0WintQrMb4s7Z0dauHnUtxwoG2vI5DkLtS3qm9Ekf"
  crossorigin="anonymous"></script>
<script>
  let url = 'https://jsonplaceholder.typicode.com/users/';
  fetch(url)
    .then(response => response.json())
    .then(data => mostrarData(data))
    .catch(error => console.log(error))
  const mostrarData = (data) => {
    console.log(data)
    let body = ""
    for (var i = 0; i < data.length; i++) {
      body += `<tr><td>${data[i].id}</td><td>${data[i].name}</td><td></td></tr>`
    }
    document.getElementById('data').innerHTML = body
    //console.log(body)
  }
</script>
```



Compara el resultado obtenido

Id	Name	Email
1	Leanne Graham	
2	Ervin Howell	
3	Clementine Bauch	
4	Patricia Lebsack	
5	Chelsey Dietrich	
6	Mrs. Dennis Schulist	
7	Kurtis Weissnat	
8	Nicholas Runolfsdottir V	
9	Glenna Reichert	
10	Clementina DuBuque	



Ejercicio 3: Uso de Fetch con Live Server

Objetivo del ejercicio:

Completar y enriquecer la visualización de datos obtenidos mediante `fetch()`, integrando múltiples propiedades del objeto JSON en una tabla HTML y comprendiendo la forma correcta de acceder a propiedades anidadas en objetos JavaScript.



Inserta email en el código

Analiza el código usado en el ejercicio 2, después, encuentra el lugar correcto dónde se deba incluir la siguiente línea de código que agrega el correo electrónico a la `tabla` `${data[i].email}`

```
for (var i = 0; i < data.length; i++) {  
    body += `<tr>  
        <td>${data[i].id}</td>  
        <td>${data[i].name}</td>  
        <td>${data[i].email}</td>  
    </tr>`;   
}
```





Verificación con Live Server

1. Guarda los cambios.
 2. Haz clic derecho sobre el HTML.
 3. Selecciona: "**Open with Live Server**".
-  El correo aparecerá en la columna correspondiente.



Id	Name	Email
1	Leanne Graham	Sincere@april.biz
2	Ervin Howell	Shanna@melissa.tv
3	Clementine Bauch	Nathan@yesenia.net
4	Patricia Lebsack	Julianne.OConner@kory.org
5	Chelsey Dietrich	Lucio_Hettinger@annie.ca
6	Mrs. Dennis Schulist	Karley_Dach@jasper.info
7	Kurtis Weissnat	Telly.Hoeger@billy.biz
8	Nicholas Runolfsdottir V	Sherwood@rosamond.me
9	Glenna Reichert	Chaim_McDermott@dana.io
10	Clementina DuBuque	Rey.Padberg@karina.biz



Agrega más columnas

Datos disponibles:

- username
- email
- city (desde address)
- phone
- website
- company.name



Código actualizado

```
<tr>
  <th>Id</th>
  <th>Name</th>
  <th>Email</th>
  <th>Username</th>
  <th>Email</th>
  <th>City</th>
  <th>Phone</th>
  <th>Website</th>
  <th>Company</th>
</tr>
```

```
body += `<tr>
  <td>${data[i].id}</td>
  <td>${data[i].name}</td>
  <td>${data[i].email}</td>
  <td>${data[i].username}</td>
  <td>${data[i].email}</td>
  <td>${data[i].address.city}</td>
  <td>${data[i].phone}</td>
  <td>${data[i].website}</td>
  <td>${data[i].company.name}</td>
</tr>`;
```



Regresamos 8:30 PM

Receso

Duración: 10 minutos.



Actividad Guiada

Parte 2

Duración: 45 minutos.



Ejercicio 4: AJAX con XML y Combobox dependientes

Objetivo del ejercicio:

Diseñar e implementar una **interfaz web** interactiva que permita consultar códigos postales por **estado** y **municipio** desde un archivo XML del Servicio Postal Mexicano, utilizando AJAX para mostrar **dinámicamente** los resultados en una **tabla ordenable**.

```
<link rel="stylesheet" href="css/style.css">  
<script src="js/main.js"></script>
```



Pasos clave

1. **Descargar el archivo XML oficial** desde:
https://www.correosdemexico.gob.mx/SSLServicios/ConsultaCP/CodigoPostal_Exportar.aspx
2. **Crear una interfaz HTML** con dos `<select>`:
 - Primer select: muestra **Estados de la República** (ordenados alfabéticamente). `<select id="estados" onchange="cargarMunicipios()">`
 - Segundo select: muestra **Municipios filtrados** según el estado seleccionado. `<select id="municipios" onchange="mostrarDatos()">`



3. Cargar datos usando **AJAX** desde el XML:

- Al seleccionar un municipio, mostrar en una tabla los datos:
Código Postal + Municipio + Estado + Ciudad + Clave de Oficina

4. Agregar opción "**TODOS**":

- Al elegir "TODOS", mostrar todos los códigos postales del estado seleccionado.

5. Ordenar la tabla:

- Agrega funcionalidad para **ordenar las columnas** al hacer clic sobre los encabezados (puede ser con JavaScript nativo o alguna función sencilla).



Componentes técnicos sugeridos

- **HTML:**

Estructura con `<select>`, `<option>`, `<table>`, `<thead>`, `<tbody>`, `<div>`.

- **CSS:** Estilos limpios, visuales claros para distinguir los datos.

- **JavaScript:**

- XMLHttpRequest o Fetch para cargar y parsear el XML.
- Uso de DOMParser para convertir texto XML en un objeto manipulable.
- Eventos onchange para cambiar dinámicamente el contenido de la tabla.
- Función para **ordenar columnas** por nombre, código postal, etc.



¿Qué competencias fortaleceremos?

- Lógica de programación y estructuras condicionales.
- Manipulación del DOM y eventos.
- Consumo y análisis de datos en formato XML.
- Mejora de la experiencia del usuario mediante interfaces dinámicas.
- Buenas prácticas con AJAX y filtros dinámicos.



Estructura del Proyecto

```
proyecto-codigos-postales/  
├── index.html  
├── css/  
│   └── style.css  
├── js/  
│   └── main.js  
├── data/  
│   └── codigos.xml
```



Importante:

- Archivo XML en carpeta /data
- CSS y JS organizados en carpetas separadas



JavaScript: Flujo lógico

1. **Cargar el archivo XML al iniciar la página** (usando XMLHttpRequest o fetch).
2. **Extraer estados únicos** → llenar primer <select>.
3. **Al seleccionar un estado**, se filtran los municipios únicos.
4. **Al seleccionar un municipio (o TODOS):**
 - Se recorre el XML para filtrar los datos por estado y municipio.
 - Se construye una tabla.
5. **Ordenamiento de columnas:** se puede agregar con un onclick al <th> de la tabla o usando una función de ordenamiento simple.



Paso 1 – Interfaz HTML

Estructura de la GUI

- `<select id="estados">` para los estados
- `<select id="municipios">` que se llena dinámicamente
- `<div id="tabla">` donde se insertará la tabla con los datos



Paso 2 – Leer XML con AJAX

let xmlDoc

```
window.onload = function () {  
  console.log("Cargando datos...")  
  fetch("data/codigos.xml")  
    .then(res => res.text())  
    .then(data => {  
      const parser = new DOMParser();  
      xmlDoc = parser.parseFromString(data, "text/xml");  
      cargarEstados();  
    })  
    .catch(error => console.error("Error al cargar el archivo XML:", error));  
};
```



Usa fetch y DOMParser para procesar el archivo XML local.



Paso 3 – Llenar Select de Estados

Función `cargarEstados()`

- Extrae todos los valores únicos de `<d_estado>`
- Llena el `<select id="estados">` con opciones **ordenadas**

```
function cargarEstados() {  
  console.log("Cargando estados...");  
  let estados = new Set();  
  let registros = xmlDoc.getElementsByTagName("table");  
  
  for(let r of registros){  
    let estado = r.getElementsByTagName("d_estado")[0].textContent;  
    estados.add(estado);  
  }  
  
  let lista = Array.from(estados).sort();  
  let select = document.getElementById("estados");  
}
```

```
lista.forEach( e => {  
  let opt = documento.createElement("option");  
  opt.value = e;  
  opt.textContent = e;  
  select.appendChild(opt);  
});
```



Paso 4 – Llenar Municipio por Estado

Función `cargarMunicipios()`

- Se ejecuta cuando cambia el estado
- Filtra los municipios (`<D_municipio>`) que pertenecen al estado seleccionado

Agrega opción `"TODOS"` para mostrar todos



Paso 5 – Mostrar datos en tabla

Función `mostrarDatos()`

- Filtra registros por **estado y municipio**
- Muestra columnas:
 - Código Postal
 - Municipio
 - Estado
 - Ciudad
 - Clave de Oficina



Se ejecuta al seleccionar municipio



Paso 6 – Tabla Ordenable

Mejora UX: Ordenar tabla

```
<th onclick="ordenarTabla(0)">Código Postal</th>
```

 Usa una función `ordenarTabla(colIndex)` para ordenar filas por columna al dar clic.



Resultado Esperado

- Combobox dinámicos con Estados y Municipios
 - Tabla interactiva que responde al cambio de selección
 - Datos obtenidos desde un XML local con fetch
-  ¡Simulación real de una consulta web sin recargar la página!



Aprendizajes Clave:

Integración de:

- HTML, CSS, JavaScript
- AJAX con fetch()
- DOM dinámico
- Lectura de XML



Cierre

Duración: 10 minutos.



Cierre de la sesión

Hoy implementamos un sistema de consulta inteligente de códigos postales usando:

- ✓ AJAX
- ✓ XML
- ✓ Selects dinámicos
- ✓ DOM y tablas interactivas
- ✓ Organización profesional del proyecto



¿Qué aprendimos hoy?

Competencias técnicas:

- Leer y procesar archivos XML con JavaScript
- Implementar fetch() y DOMParser
- Crear interfaces reactivas (GUI) con HTML/JS
- Organizar un proyecto como en entornos reales

Habilidades prácticas:

- Depuración de errores en consola
- Validación de rutas, estructura y nodos
- Trabajo paso a paso con enfoque en el detalle